Task Management System — Documentation & Theoretical Case

Task Management System

A Task Management web application built with **Next.js 15 (App Router)**, **tRPC v11**, and in-memory storage.

Table of Contents

1. Part 1 — Theoretical Questions
   - 1. ReactJS & Next.js: Rendering Strategies
   - 2. Backend Concepts, Approaches & tRPC Analysis
2. Part 2 — Implementation & Architecture
   - Architecture Decisions
   - Data Model
   - Validation & Error Handling
3. Running Locally
4. Testing Suite
5. Bonus Features

Part 1 — Theoretical Questions

1. ReactJS & Next.js: Rendering Strategies

a) Core Differences Between CSR, SSR, and SSG

- **Client-Side Rendering (CSR)**:
  - **Mechanism**: The web server delivers a minimal HTML shell (e.g. `<div id="root"></div>`) and JavaScript bundles. The browser downloads and executes the JavaScript, requests dynamic data asynchronously from backend APIs, and constructs the DOM tree directly on the client.
  - **Trade-off**: Offloads rendering compute to user devices, but incurs a cold-start waterfall (downloading JS → execution → API request → layout paint), leading to slower First Contentful Paint (FCP) and fragile SEO indexing.
- **Server-Side Rendering (SSR)**:
  - **Mechanism**: On **every incoming HTTP request**, the server executes component render logic, fetches required database/API resources, renders the React tree into full HTML, and streams or returns that HTML to the browser. Once rendered, React hydrates the DOM on the client to attach event listeners and interactive state.
  - **Trade-off**: Guarantees up-to-the-second data freshness and fast First Contentful Paint with complete HTML on the first byte, but increases server compute load and Time to First Byte (TTFB) compared to static caching.
- **Static Site Generation (SSG)**:

- **Mechanism**: HTML and JSON payloads are pre-rendered once at **build time** (`next build`). Pages can optionally be re-generated in the background via Incremental Static Regeneration (ISR). The generated static assets are hosted directly on global CDN edge nodes.
- **Trade-off**: Delivers near-zero TTFB and scales globally with minimal compute costs, but is ill-suited for per-request personalized content or rapidly mutating transactional datasets.

Strategy	Where HTML is Built	TTFB (Time to First Byte)	FCP & SEO Readiness	Data Freshness	Compute Scaling
CSR	Client Browser	Fast (empty shell)	Slow (JS execution delay)	Live (client fetch on mount)	Zero server compute
SSR	Server (per request)	Dependent on backend fetch	Fast (populated HTML)	Always fresh per request	Scales with traffic
SSG	Build Server / Edge	Instant (CDN cached)	Fast (pre-rendered)	Static until rebuild / ISR	Minimal edge compute

b) Practical Scenarios: Advantageous vs. Problematic

1. Client-Side Rendering (CSR)

- **Advantageous**: Authenticated, state-heavy interactive web applications where SEO is irrelevant and users remain in long continuous sessions.
  - *Examples*: Figma canvas editors, Linear project boards after the initial shell load, internal admin metrics dashboards, WebRTC meeting rooms.
- **Problematic**: Public marketing pages or e-commerce landing pages that rely on search engine crawlers and social media OpenGraph previews, or mobile users on slow networks where multi-megabyte JS bundles cause layout shifts and blank loading states.

2. Server-Side Rendering (SSR)

- **Advantageous**: Applications requiring **both** real-time / user-personalized data **and** immediate content paint without client fetch waterfalls.
  - *Examples*: E-commerce product details with dynamic inventory and real-time pricing, personalized social feeds, task management dashboards (like this project) where the initial list must arrive pre-rendered on the first byte.
- **Problematic**: High-traffic public content that rarely changes (running SSR on every request wastes server CPU and exhausts backend database connection pools), or pages dependent on slow third-party APIs that block HTML delivery without Suspense streaming.

3. Static Site Generation (SSG)

- **Advantageous**: Content shared identically across all visitors that updates predictably or infrequently.
  - *Examples*: Technical documentation portals, developer blogs, marketing landing pages, company knowledge bases.
- **Problematic**: Applications with hundreds of thousands of dynamic records that mutate constantly (triggering long build times) or pages requiring request headers, geolocation, or session cookies at render

time.

---

2. Backend Concepts, Approaches & tRPC Analysis

a) Main Approaches to Implementing a Web Application Backend

1. **RESTful APIs (HTTP + JSON / OpenAPI)**:
   - Resource-oriented architecture using standard HTTP verbs (`GET`, `POST`, `PUT`, `PATCH`, `DELETE`) and status codes. Universally understood, language-agnostic, and easily cached at CDN/HTTP proxy boundaries.
2. **GraphQL**:
   - A schema-driven query language over a single endpoint where clients declare the exact shape of nested fields they require. Solves over-fetching and under-fetching across complex graphs, with trade-offs in query complexity analysis, N+1 resolver caching (DataLoader), and non-trivial HTTP caching.
3. **RPC & BFF (Remote Procedure Call / Backend-for-Frontend — e.g., tRPC, gRPC)**:
   - Action/procedure-oriented communication where clients invoke remote procedures as if calling local functions. `gRPC` dominates low-latency polyglot microservice communication via Protocol Buffers over HTTP/2, while `tRPC` provides an ergonomic, zero-codegen TypeScript BFF layer.
4. **Event-Driven & Real-Time Architectures (WebSockets, SSE, Message Queues)**:
   - Asynchronous pipelines (Kafka, RabbitMQ, Redis Streams) or persistent bidirectional sockets (WebSockets, Server-Sent Events) designed for live collaborative editing, telemetry feeds, AI token streaming, and background job processing.

b) Deep Dive on tRPC: Security, Type Safety, and Scalability

tRPC (TypeScript Remote Procedure Call) enables full-stack TypeScript applications to share backend procedure signatures directly with the frontend through TypeScript's static type inference (`import type { AppRouter }`), eliminating manual OpenAPI specs or code-generation pipelines.

Key Advantages:

- **End-to-End Type Safety**: Changes in backend inputs or return structures immediately surface as compile-time TypeScript errors across frontend consumers. Refactoring a field name in a router takes seconds without API drift.
- **Security & Runtime Validation**: Because TypeScript types disappear at runtime, tRPC pairs natively with **Zod**. Every incoming payload is validated at the network boundary before reaching domain logic. Context and composable middleware pipelines (`publicProcedure`, `protectedProcedure`) enforce centralized authentication, role-based authorization, and rate limiting.
- **Developer Velocity & Performance**: Zero build-step codegen overhead, native HTTP request batching (combining concurrent queries in a single React render into one HTTP roundtrip), and first-class integration with **TanStack React Query** and **Next.js 15 Server Components (RSC Hydration)**.

Key Disadvantages & Scalability Trade-offs:

- **Polyglot & Multi-Platform Lock-in**: tRPC requires **both** server and client to run TypeScript. If the team expands into native mobile apps (`Swift` for iOS / `Kotlin` for Android), data science services in `Python`, or high-throughput services in `Go` / `Rust`, non-TS clients cannot consume tRPC types natively (requiring an adapter like `trpc-to-openapi` or a dedicated REST/gRPC gateway).

- **Repository Coupling**: Works effortlessly in a unified monorepo (Next.js, Turborepo, Nx), but introduces packaging friction across segregated multi-repo team structures where types must be published to a private registry.
- **Public API Limitations**: tRPC is optimized for internal client-server communication (BFF), not for public developer platforms that require traditional OpenAPI/Swagger documentation and standard REST caching semantics.

Part 2 — Implementation & Architecture

```
src/
├── server/
│   ├── db/
│   │   └── task-store.ts        # In-memory singleton store with initial seed tasks
│   └── api/
│       ├── trpc.ts              # tRPC context, SuperJSON transformer & Zod errorFormatter
│       ├── root.ts              # AppRouter definition & createCallerFactory
│       ├── routers/
│       │   └── task.ts          # CRUD procedures with Zod validation & semantic TRPCError
│       └── routers/__tests__/
│           └── task.test.ts     # Vitest integration test suite
├── trpc/
│   ├── query-client.ts          # QueryClient factory with SuperJSON dehydration
│   ├── client.tsx               # TRPCReactProvider for Client Components
│   └── server.ts                # RSC caller & HydrateClient for Next.js 15 SSR
├── app/
│   ├── api/trpc/[trpc]/route.ts # Next.js App Router fetch adapter endpoint
│   ├── layout.tsx               # Root layout with Provider & Sonner Toaster
│   ├── page.tsx                 # SSR Task List Page (prefetchInfinite + HydrateClient)
│   ├── tasks/
│   │   ├── new/page.tsx         # Task Creation Page
│   │   └── [id]/edit/page.tsx   # Task Edit Page (SSR prefetch by ID)
│   └── not-found.tsx            # 404 error boundary
└── components/
    ├── task-list.tsx            # Hydrated list with Infinite Scroll & direct deletion
    └── task-form.tsx            # Reusable functional form with React hooks & validation
```

Architecture Decisions

1. **In-Memory Store Persistence (`globalThis` Singleton)**: In Next.js App Router (especially during development Fast Refresh or between Server Component Callers and Route Handlers), plain module variables can be re-evaluated. Attaching the `InMemoryTaskStore` instance to `globalThis` preserves the state across requests without accidental resets.
2. **Server-Side Rendering (SSR) Hydration**: The listing page (`src/app/page.tsx`) uses `export const dynamic = "force-dynamic"` and prefetches the initial task page on the server via `trpc.task.list.prefetchInfinite({ limit: 8 })`. The HTML is rendered on the server with populated data and hydrated on the client via `<HydrateClient>` with zero initial loading flash.
3. **Cursor-Based Infinite Scroll**: The `task.list` procedure supports `{ limit, cursor }` pagination. The client consumes `useInfiniteQuery` and an `IntersectionObserver` sentinel to automatically load subsequent task pages as the user scrolls.
4. **Direct Deletion with Feedback**: Tasks can be deleted directly from the listing view with pending indicators and Sonner toast notifications.

Data Model

```
export interface Task {
  id: string;          // Unique identifier (UUID v4 / deterministic seed)
  title: string;       // Mandatory task title (1..120 chars)
  description?: string;// Optional task description (max 1000 chars)
  createdAt: Date;     // Creation timestamp
}
```

Validation & Error Handling

- **Zod Runtime Validation**: Input titles are validated with `.trim().min(1)` to strictly prevent whitespace-only or empty submissions.
- **Semantic `TRPCError`**: Operations targeting non-existent IDs (such as `task.byId`, `task.update`, or `task.delete`) throw `TRPCError` with code `NOT_FOUND` and descriptive error messages.
- **Frontend Error Feedback**: Forms display error banners and listing actions trigger toast notifications.

Running Locally

1. Install Dependencies

```
npm install
```

2. Start Development Server

```
npm run dev
```

Open http://localhost:3000 in your browser.

3. Build for Production

```
npm run build
npm run start
```

Testing Suite

Run the automated integration test suite with Vitest:

```
npm test
```

The test suite (`src/server/api/routers/__tests__/task.test.ts`) verifies: * Cursor-based pagination and descending order by `createdAt`. * Retrieval of tasks by ID and `NOT_FOUND` error handling for invalid IDs. * Task creation with automatic ID/timestamp generation and rejection of empty titles. * Task updating and rejection of non-existent task IDs. * Task deletion and verification of subsequent `NOT_FOUND` lookup.

Bonus Features

1. **Infinite Scroll**: Cursor-based pagination using `IntersectionObserver` on the task list.
2. **Dedicated Pages**: `/tasks/new` (Creation) and `/tasks/[id]/edit` (Update) routes with deep linking, SSR lookup, and clean 404 boundaries.
3. **Automated Integration Tests**: Full test suite built with Vitest.